

Code Explanation

The `segregateCorrectly` function takes an array of numbers and rearranges it so that all negative numbers come before all non-negative numbers (zero or positive). It uses a technique called the two-pointer approach.

Here's how it works:

1. Initialization:
 - `left` pointer starts at the beginning of the array (index 0).
 - `right` pointer starts at the end of the array (last index).
2. Main Loop (`while (left < right)`): The core logic runs as long as the `left` pointer is to the left of the `right` pointer.
 - Finding a Misplaced Non-negative Number (Left Pointer):
 - The inner `while (left < right && arr[left] < 0)` loop moves the `left` pointer forward as long as it encounters negative numbers. It stops when it finds a non-negative number (`arr[left] >= 0`) or when `left` crosses `right` . This non-negative number is "misplaced" if it's on the left side.
 - Finding a Misplaced Negative Number (Right Pointer):
 - The inner `while (left < right && arr[right] >= 0)` loop moves the `right` pointer backward as long as it encounters non-negative numbers. It stops when it finds a negative number (`arr[right] < 0`) or when `right` crosses `left` . This negative number is "misplaced" if it's on the right side.
 - Swapping (`if (left < right)`):
 - If, after both inner loops, `left` is still less than `right` (meaning they haven't crossed), it implies that `arr[left]` is a non-negative number that should be on the right, and `arr[right]` is a negative number that should be on the left.
 - These two elements are then swapped.
 - After the swap, both `left` and `right` pointers are moved one step inward (`left++` , `right--`) to continue the search for other misplaced elements.
3. Return: Once `left` becomes greater than or equal to `right` , the main loop terminates, and the array is correctly segregated. The function then returns the modified array.

Tracing Example

Let's trace the function with the input array: `arr = [-1, 2, -3, 4, -5, 6]`

Initial State:

```
arr = [-1, 2, -3, 4, -5, 6]
left = 0 , right = 5
```

Iteration 1:

- Main loop condition: `left (0) < right (5)` is true.
- Move `left`: `arr[0]` is `-1` (negative), so `left` becomes `1` .
 - `arr[1]` is `2` (non-negative). `left` stops at `1` .
 - Current state: `left = 1` , `right = 5`

- Move `right` : `arr[5]` is 6 (non-negative), so `right` becomes 4 .
 - `arr[4]` is -5 (negative). `right` stops at 4 .
 - Current state: `left` = 1 , `right` = 4
- Swap condition: `left` (1) < `right` (4) is true.
- Swap `arr[1]` (2) and `arr[4]` (-5):
 - `arr` becomes [-1, -5, -3, 4, 2, 6]
- Move pointers: `left` becomes 2 , `right` becomes 3 .
- End of Iteration 1.

Iteration 2:

- Main loop condition: `left` (2) < `right` (3) is true.
- Move `left` : `arr[2]` is -3 (negative), so `left` becomes 3 .
 - `arr[3]` is 4 (non-negative). `left` stops at 3 .
 - Current state: `left` = 3 , `right` = 3
- Move `right` : `arr[3]` is 4 (non-negative), so `right` becomes 2 .
 - Current state: `left` = 3 , `right` = 2
- Swap condition: `left` (3) < `right` (2) is false. Pointers have crossed (`left` is no longer less than `right`).
- End of Iteration 2.

Final State:

- Main loop condition: `left` (3) < `right` (2) is false. The loop terminates.
- The function returns `arr` = [-1, -5, -3, 4, 2, 6] .

Illustration of Pointer Movements

Here's a visual representation of how the `left` and `right` pointers move and how the array changes. [^]L indicates the `left` pointer's position, and [^]R indicates the `right` pointer's position.

Initial Array:

```
arr = [-1, 2, -3, 4, -5, 6]
^L           ^R
```

Iteration 1:

- Initial state:


```
[-1, 2, -3, 4, -5, 6]
^L           ^R
```
- `left` moves past -1 (stops at 2):


```
[-1, 2, -3, 4, -5, 6]
^L           ^R
```
- `right` moves past 6 (stops at -5):


```
[-1, 2, -3, 4, -5, 6]
^L           ^R
```
- Swap `arr[left]` (2) and `arr[right]` (-5):


```
arr becomes [-1, -5, -3, 4, 2, 6]
```

^L ^R

5. Move pointers inward after swap:

[-1, -5, -3, 4, 2, 6]

^L ^R

Iteration 2:

1. Current state:

[-1, -5, -3, 4, 2, 6]

^L ^R

2. left moves past -3 (stops at 4):

[-1, -5, -3, 4, 2, 6]

^L ^R

3. right moves past 4 (stops at 2, but crosses left):

[-1, -5, -3, 4, 2, 6]

^R ^L

4. Pointers Cross:

At this point, left (3) is no longer less than right (2). The condition `left < right` becomes false, and the loop terminates. The array is now segregated.

Final Segregated Array:

[-1, -5, -3, 4, 2, 6]